No.

10 - dars:

▲ target_col = "goals"

drop_cols = ["id", "player_name"] # id + player_name
Non-generalizable identifiers

```
X = df.drop(columns = [target_col] + drop_cols).copy()
y = df[target_col].copy()
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size = 0.2 , random_state = 42
)
```

```
num_cols = X_train.select_dtypes(include = ["int64",
                                  "float64"]).columns.tolist()
cat_cols = X_train.select_dtypes(include = ["object"]).
                                  columns.tolist()
```

```
for c in num_cols:
    mean_val = X_train[c].mean()
    X_train[c] = X_train[c].fillna(mean_val)
    X_test[c] = X_test[c].fillna(mean_val)
```

No.

```
for c in cat_cols:
    mode_series = X_train[c].mode(dropna=True)
    mode_val = mode_series.iloc[0] if len(mode_series) else
                                        "MISSING"
    X_train[c] = X_train[c].fillna(mode_val)
    X_test[c] = X_test[c].fillna( mode_val)
```

Encoding

```
onehot_cols = []
label_cols = []

for c in cat_cols:
    n_unique = X_train[c].nunique(dropna=True)
    if n_unique <= 5:
        onehot_cols.append(c)
    else:
        label_cols.append(c)

print("One-hot columns (<=5 classes):", onehot_cols)
print("Label-encoded columns (>5 classes):", label_cols)

X_train_enc = X_train[num_cols].copy()
X_tst_enc = X_test[num_cols].copy()
```

Label encode each column using Train categories

```
for c in label_cols:
    X_train_enc[c] = X_train[c].astype("category").cat.
                                                codes
```

No.

```
# Use the SAME categories from train; unseen in test → -1
cats = X_train[c].astype("category").cat.categories
X_test_enc[c] = pd.Categorical(X_test[c], categories
                               = cats).codes

# One-hot encode (train/test separately)
X_train_oh = pd.get_dummies(X_train, columns = onehot_cols)
X_test_oh = pd.get_dummies(X_test, columns = onehot_cols)

# Make columns match
X_train_oh, X_test_oh = X_train_oh.align(X_test_oh,
                        join = "left", axis = 1, fill_value = 0)

print("\n Final Encoded Shapes:")
print("X_train_enc:", X_train_enc.shape)
print("X_test_enc:", X_test_enc.shape)

model = RandomForestRegressor(
    n_estimators = 400,
    random_state = 42,
    n_jobs = -1
)
model.fit(X_train_enc, y_train)

preds = model.predict(X_test_enc)
print("\n MAE:", round(mean_absolute_error(y_test, preds), 4))
print("R2:", round(r2_score(y_test, preds), 4))
```

No.

Linear Regression

```
from sklearn.linear_model import LinearRegression
model_linear = LinearRegression()
model_linear.fit(X_train_enc, y_train)
```

▲ explainer = shap.Explainer(model, X_train_enc, feature_names = X_train_enc.columns.tolist())

! Explainer => Bu = 2 xili bor
Tree Explainer, Linear Explainer => Bu dasiga qarab farq qiladi va ishlatilinadi.

Linear xolat uchun: shap.LinearExplainer()
Non-linear xolat/ u-n: shap.Explainer() yoki shap.TreeExplainer()

# Oz qiymat b/u tekshir sabab dataset katta vaqt kop ketadi
X_sample = X_test_enc.iloc[:500] # istalgancha
shap_values = explainer(X_sample, check_additivity = False)

Global Shap

```
shap.plots.bar(shap_values, max_display=20)
plt.show()
```

Odatda prediction[0] = Baseline + Sum(Shap) Bo'ladi. Agar teng bo'lmasa (error) berib chizma chiqmaydi. Agar check_additivity = False ni yozmasak. Aslida 99.9% da xatoda to'g'ri chiqadi osha 0.01% holatni oldini olish uchun yozib ketamiz

▲ shap.values.base_values gilsak baseline value/ini ko'rsak bo'ladi

~~Waterflo~~ Summary_plot. B/n ham topish

▲ shap.summary_plot(
    shap_values.values,
    X_sample,
    feature_names = X_sample.columns.tolist(),
    max_display = 20
)
plt.show()

local SHAP (waterfall)

idx = 0
pred_one = model.predict(X_sample.iloc[idx:idx+1])[0]
print("\n local sample idx:", idx, "| Predicted goals:", pred_one)
shap.plots.waterfall(shap_values[idx], max_display=15)
plt.show()

No.

Fairness Checks & Bias Mitigation

- Model accurate ishlayapti:
  - ↳ To'g'ri mantiqli prediction
  - ↳ Yaxshi accuracy

- Shu xolatda ham modelni yaxshi deya olamizmi?

⇓

Fairness checks tekshirish k-k

Bazi hollarda modellar baised modellar (tarafkash) bo'lishi mumkin.

Masalan; odamlarga zarar yetadigan holat bo'lishi m-n (harm people), yoki ishonchga putur yet kazishi mumkin (~~Doge~~ Damage trust), yoki turli xil noto'g'ri xolat/ga & business xisklarga olib kelishi m-n. (Cause legal & business risks) modellar bo'lib qolishi m-n. Lekin modellar zo'r va to'g'ri ishlayapti. Lekin biz fairness checks qilmasligimiz oqibatidan 3 ta yomon holatga olib kelishi m-n.

No.

Example

- Ikki sportchi: bir xil sharoit, murabbiy lekin biri tabiattan kuchsiz shunga past ball oladi.

↳ Sababi boshida bu = Baseline ni past bo'lganligi u-n.

Fairness check:

- Adolatsizlikni aniqlash, kuzatish va tushuntirish
- Malum bir guruh doimiy past va noto'g'ri chiqayapti => bu shubhalantiradigan holat

qiymat berib tekshiramiz va ha bu yerda fairness checks buzilayapti deymiz, buni to'g'irlashimiz kerak deymiz.

- Diagnosis (Tashxis) qo'yish

Bias mitigation

Diagnosis (Tashxis) qo'yish --- Fairness checks

Bias Mitigation -- Davolash

⇓

3ta qismdan iborat

Data level

↳ Balance datasets

↳ Re-sample underrepresented groups

↳ Improve data collection, Clean biased labels

↳ Remove or redesign sensitive features

No.

- Model - level
  - ↳ Add fairness constraints (cheklov qōyish) = Threshold
  - ↳ Penalize reliance on sensitive features

- Post - processing level
  - ↳ Training bōlgandan keyin natijalar ustida ishlanadi va adolatli natijaga erishiladi.

Masalan: Kredit berish chegarasi ~~...~~ umumiy 0.7 bōlsa kōpchillik olololmasligi mumkin, ba'zi esa ba'zi guruhlar u-n kamaytiramiz masalan 0.65 qilishimiz mumkin.